Documentación Técnica

Documentación Técnica — Algoritmos Iterativos vs. Recursivos

Nombre: Diego Lima **Asignatura:** Estructura de Datos **Catedrático:** Ing. Brandon Chitay **Universidad:** Da Vinci de Guatemala **Fecha:** Marzo 2026

1. ¿Qué es Big-O?

Big-O mide cómo crece el tiempo de un algoritmo cuando la entrada (n) aumenta:

- **O(n)** — si n se duplica, el tiempo se duplica.
- **O(n^2)** — si n se duplica, el tiempo se cuadruplica.
- **O(2^n)** — cada incremento de n duplica el tiempo.

2. Algoritmos implementados

Factorial — Calcula n! (ejemplo: 5! = 120). El iterativo usa un ciclo acumulador; el recursivo se llama con `n * fact(n-1)`. Ambos son **O(n)**.

Fibonacci — Calcula el n-ésimo término (0, 1, 1, 2, 3, 5, 8…). El iterativo usa dos variables que se desplazan en un ciclo. El recursivo llama a `fib(n-1) + fib(n-2)`, repitiendo cálculos y generando un árbol exponencial. Iterativo: **O(n)**, recursivo: **O(2^n)**.

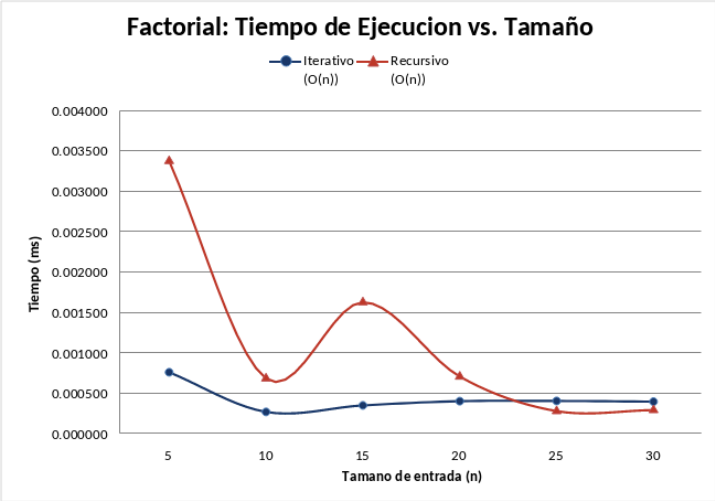
Búsqueda Lineal — Busca un valor recorriendo el arreglo. El iterativo usa un `for`; el recursivo avanza el índice en cada llamada. Ambos son **O(n)**.

Burbuja — Ordena comparando e intercambiando pares adyacentes. El iterativo usa dos ciclos anidados; el recursivo hace una pasada por llamada y reduce n en 1. Ambos son **O(n^2)**.

---

3. Gráficas de rendimiento

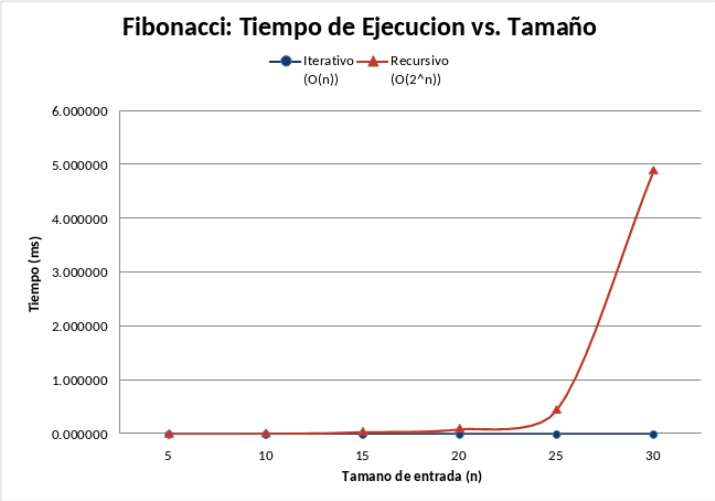
Factorial



Factorial

Ambas curvas se mantienen planas (tiempos < 0.005 ms). Confirma **O(n)** en ambas versiones.

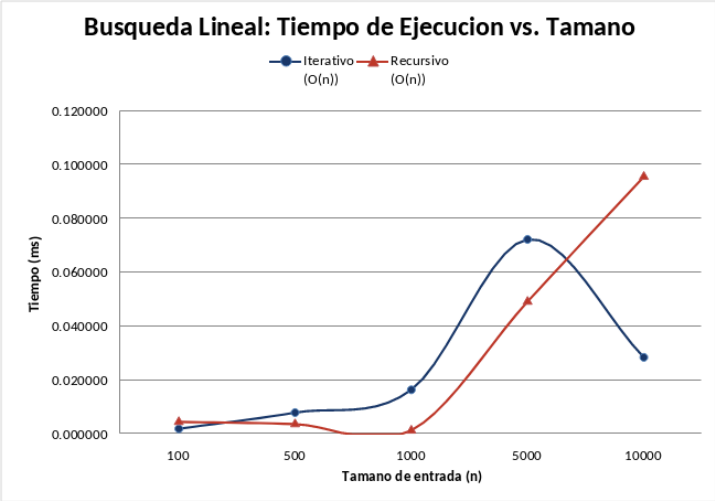
Fibonacci



Fibonacci

El iterativo se mantiene cerca de cero. El recursivo explota a partir de n=25 y llega a ~5 ms en n=30. Esa curva exponencial confirma **O(2^n)**.

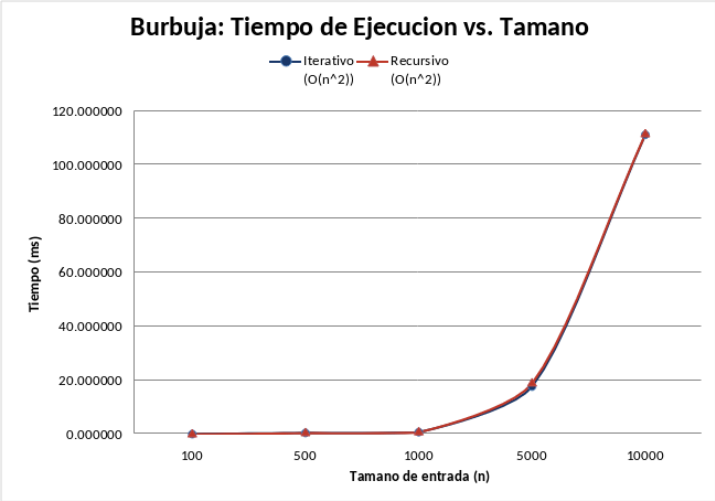
Búsqueda Lineal



Búsqueda Lineal

Ambas versiones crecen de forma ascendente conforme n aumenta. Confirma **O(n)**.

Burbuja



Burbuja

Ambas curvas son casi idénticas y forman una parábola: de n=1000 (~1 ms) a n=10000 (~110 ms). Confirma **O(n²)**.

---

4. Conclusiones — Big-O experimental vs. teórico

Algoritmo	Versión	Big-O experimental	Big-O teórico	¿Coincide?
Factorial	Iterativo	O(n)	O(n)	Sí
Factorial	Recursivo	O(n)	O(n)	Sí
Fibonacci	Iterativo	O(n)	O(n)	Sí
Fibonacci	Recursivo	$O(2^n)$	$O(2^n)$	Sí
Búsqueda Lineal	Iterativo	O(n)	O(n)	Sí
Búsqueda Lineal	Recursivo	O(n)	O(n)	Sí
Burbuja	Iterativo	$O(n^2)$	$O(n^2)$	Sí
Burbuja	Recursivo	$O(n^2)$	$O(n^2)$	Sí

Todos los resultados experimentales coinciden con la teoría.

---

5. Preguntas de análisis

¿Por qué Fibonacci recursivo es mucho más lento que el iterativo? Porque cada llamada genera dos llamadas más, creando un árbol que repite los mismos cálculos. `fib(30)` hace ~2.7 millones de llamadas. El iterativo solo necesita 30 pasos.

¿Qué significa $O(n^2)$ y cómo se ve en Burbuja? Que el tiempo crece con el cuadrado de n. En la gráfica se ve como una curva que sube cada vez más rápido: 10x más datos = 100x más tiempo.

Si n se duplica, ¿cuánto aumenta el tiempo? - $O(n)$: se duplica (×2) - $O(n^2)$: se cuadruplica (×4) - $O(2^n)$: crece de forma inviable

¿Cuál es el costo de la recursión en memoria? Cada llamada recursiva agrega un frame al call stack. Con n muy grande se produce un `StackOverflowError`. El iterativo usa memoria constante $O(1)$.

¿Cuándo usar recursivo vs. iterativo? - Recursivo: problemas que se dividen naturalmente en subproblemas (árboles, grafos). Código más corto y legible. Ejemplo: explorar las carpetas de una computadora — cada carpeta puede contener más carpetas adentro, así que la función se llama a sí misma por cada subcarpeta que encuentra. - Iterativo: cuando n es grande, se necesita eficiencia en memoria, o la recursión repite trabajo (como Fibonacci sin memoización). Ejemplo: sumar los precios de un carrito de compras — solo se recorre la lista de productos una vez con un ciclo, no tiene sentido usar recursión.

---

6. Referencias

- Cormen, T. et al. *Introduction to Algorithms*, 4th Edition. MIT Press.
- Oracle. *Java SE Documentation — System.nanoTime()*
- Sedgewick, R. & Wayne, K. *Algorithms*, 4th Edition. Addison-Wesley.